**PANAHON**

Sistema de gestión de fichajes de empleados

# Auditoría de Seguridad & Plan de Rebuild

Informe técnico y propuesta de trabajo

| | |
|---|---|
| **Fecha:** | 20 de octubre de 2026 |
| **Preparado por:** | Agustín Clavero |
| **Para:** | Matias Godoy |
| **Alcance:** | Seguridad, frontend y plan de reconstrucción |
| **Estado:** | Preliminar — a confirmar con el código fuente |

*Documento confidencial. Elaborado para uso del cliente y su equipo técnico.*

# 1 · Resumen ejecutivo

Panahon es un sistema de fichajes multiempresa, armado con Figma Make sobre Supabase. La parte funcional está bastante bien: tiene geovalla por GPS, marca los registros sospechosos, deja rastro de las ediciones manuales, guarda los borrados y avisa con alertas.

El problema está en dos puntos de fondo. Uno: al fichar no se verifica quién es la persona, así que cualquiera puede marcar la entrada de otro. Dos: el frontend lee la base directamente desde el navegador, con lo cual toda la seguridad termina dependiendo de cómo esté configurado RLS en Supabase.

El diseño visual se puede aprovechar; lo que hay que rehacer es el código. La idea es reconstruir el frontend en un stack profesional (Next.js), que de paso ordena el acceso a los datos, y sumar el fichaje con biometría (WebAuthn) para que no se pueda fichar por otro.

# 2 · Alcance y metodología

Para la revisión usé la cuenta de prueba de Panahon y las credenciales que me pasaron (el panel de administración y una sesión activa). Trabajé mirando cómo se comporta la app en vivo y qué pedidos le hace a la base mientras se usa. No toqué ni borré nada, y no registré ningún fichaje.

Hay algunos puntos que recién voy a poder confirmar cuando tenga el código (el repositorio de GitHub, todavía pendiente). Esos quedan marcados como *“a confirmar”*.

# 3 · Arquitectura detectada

- **Backend:** Supabase (PostgreSQL + PostgREST + Edge Functions). Existe una Edge Function (`make-server-bdf30ab0`) que atiende escrituras y operaciones de administración.
- **Lecturas:** el frontend consulta la base **directamente desde el navegador** vía `/rest/v1/...` con la clave pública, filtrando por `empresa_id` del lado del cliente.
- **Multiempresa:** cada negocio aislado por `empresa_id`; plan/suscripción y pagos vía MercadoPago.
- **Fichaje:** página pública (PWA) accesible por un QR con el id de empresa en la URL; geovalla GPS configurable.
- **Frontend:** SPA de React autogenerado por Figma Make.

# 4 · Hallazgos de seguridad

**SEVERIDAD CRÍTICA**

**C1 · Fichaje sin verificación de identidad (suplantación / “buddy punching”)**

La página pública de fichaje no solicita autenticación: el empleado se registra simplemente **eligiéndose de una lista**. El único control es la geovalla GPS, que prueba la ubicación, no la identidad.

***Evidencia:*** */fichaje?empresa=<uuid> muestra la pantalla “¿Quién sos?” con la lista de empleados y permite fichar sin credenciales.*

**Impacto:** Cualquiera puede fichar por un compañero ausente; el GPS además es falsificable desde el navegador. Invalida la confiabilidad de todos los registros horarios.

### C2 · Padrón de empleados expuesto públicamente

La misma página pública lista los **nombres reales de todos los empleados** a cualquiera que abra el enlace, que viaja en un QR descargable e impreso en el local.

***Evidencia:*** *Listado completo de los 25 empleados visible sin autenticación.*

**Impacto:** Exposición de datos personales de terceros (Ley 25.326 de Protección de Datos Personales).

### C3 · Lecturas directas a la base desde el navegador — todo depende de RLS

Datos sensibles se solicitan directamente desde el cliente: empleados (con email, valor hora, sueldo y device_id), empresa completa, perfiles de administrador (con rol) y registros.

***Evidencia:*** *GET /rest/v1/empleados?select=...,valor_hora,sueldo_base_mensual,device_id · /rest/v1/empresas?select=* · /rest/v1/perfiles_admin?select=...,rol*

**Impacto:** Si las políticas de RLS están incompletas o mal definidas, un usuario autenticado —o incluso cualquiera con la clave pública, que viaja en el JavaScript— podría leer o modificar datos de **todas las empresas**. *Veredicto final a confirmar con las migraciones del repositorio.*

### C4 · Rol de administrador manejado del lado del cliente — posible escalación de privilegios

La tabla `perfiles_admin` (con el campo `rol`) se lee directamente desde el frontend.

**Impacto:** Si RLS permite a un usuario actualizar su propia fila, podría auto-asignarse rol de administrador. *A confirmar con el código.*

## SEVERIDAD ALTA

### A1 · Datos salariales enviados al navegador

La consulta de empleados devuelve `valor_hora` y `sueldo_base_mensual` al cliente. Si RLS no restringe por rol, un empleado común podría ver la remuneración de todos.

### A2 · Vínculo de dispositivo débil (device_id)

El `device_id` como mecanismo de vinculación es fácilmente copiable y no constituye prueba de identidad. Se reemplaza por credenciales WebAuthn.

### A3 · Integridad de la hora de fichaje — a confirmar

Debe verificarse que la hora del fichaje la fije el **servidor** y no el dispositivo; de lo contrario, es manipulable.

### A4 · Ausencia de cabeceras de seguridad HTTP

No se detectaron cabeceras estándar (CSP, HSTS, X-Frame-Options, X-Content-Type-Options, Referrer-Policy, Permissions-Policy), lo que facilita ataques de clickjacking, XSS y downgrade. *A confirmar.*

## SEVERIDAD MEDIA

### M1 · Identificador de empresa expuesto en URL/QR

El id público de empresa viaja en la URL de fichaje; si se filtra, da acceso al padrón de esa empresa.

### M2 · Identificadores internos expuestos en el frontend

UUIDs de empleados, empresa y administrador visibles en el cliente (divulgación de información menor).

### M3 · Telemetría (endpoint /eventos)

Revisar qué información registra el endpoint de eventos para evitar almacenar datos personales innecesarios.

## LO QUE ESTÁ BIEN RESUELTO

- Existe un **backend real** (Edge Function) para escrituras y operaciones de administración.
- Buen andamiaje anti-fraude: **geovalla GPS**, detección de **registros sospechosos**, marca de **edición manual**, **soft-delete** con historial y **alertas de seguridad**.

- Validación centralizada de estado y suscripción mediante RPC (`verificar_estado_empresa_empleado`).
- Diseño visual logrado (modo oscuro, jerarquía clara) y UX cuidada en la PWA de fichaje.

## PENDIENTE DE CONFIRMAR CON EL CÓDIGO

- Estado real de RLS por tabla (cierra C3, C4 y A1).
- Si la hora de fichaje la fija el servidor (A3).
- Si la Edge Function valida rol y empresa del lado del servidor o reenvía con la service key.
- Que no exista una `service_role key` commiteada en el repositorio (sería un hallazgo crítico adicional).

## 5 · Evaluación del frontend actual

El diseño está bien. Modo oscuro prolijo, buena jerarquía y una pantalla de fichaje con una UX cuidada. Todo eso se puede reutilizar como referencia.

El tema es cómo está hecho por dentro. Figma Make genera un SPA de React automático, y eso trae varios dolores de cabeza:

- Código difícil de leer, mantener y extender.
- **Lógica de datos mezclada dentro de los componentes** — es la raíz del problema de seguridad (acceso directo a la base desde el navegador).
- Sin separación capa-de-datos / UI, sin tipado estricto, sin pruebas; routing y estado improvisados.
- SPA sin render de servidor: no hay dónde ubicar lógica sensible; performance y SEO limitados.
- Montar el módulo biométrico y los roles sobre este código sería frágil y costoso.

Dicho corto: el diseño sirve, el código no. Se rehace la implementación, no el diseño.

## 6 · Recomendación técnica

Lo que recomiendo es rehacer el frontend en **Next.js** (App Router) con TypeScript, Tailwind y shadcn/ui, usando TanStack Query para los datos, y dejando **Supabase** como backend (Auth, base y Edge Functions).

Por qué este camino para este caso puntual:

- Con Next.js la lógica sensible y las lecturas pasan a correr en el servidor, no en el navegador. Eso corta el problema de fondo: se terminan las consultas directas con la clave pública. **Es la razón principal.**
- **TypeScript** con tipos generados desde el esquema de Supabase: menos errores y refactors seguros.
- **shadcn/ui + Tailwind**: permite reconstruir el look actual de forma rápida, profesional y mantenible.
- **TanStack Query**: caché, estados de carga y revalidación reales.
- **PWA + WebAuthn**: la biometría encaja de forma natural; requiere HTTPS, ya disponible.
- **Se conserva Supabase**: no se descarta la inversión del cliente; lo que se corrige es *cómo* se accede a los datos.

**Alternativa más liviana:** Vite + React + TS + Tailwind + shadcn como SPA. Más simple, pero obliga a colocar *toda* la lógica sensible en Edge Functions. Para este proyecto se prefiere Next.js porque aporta la seguridad por arquitectura.

Lo que no haría: seguir en Figma Make ni rehacer el backend de cero. Supabase está bien; lo que falla es cómo se accede a los datos.

## 7 · Módulo de fichaje biométrico (WebAuthn)

Esto ataca directo a C1 y C2. Se apoya en **WebAuthn / Passkeys**, el estándar web que usa el sensor del teléfono (huella, Face ID o el PIN del equipo, según lo que tenga cada uno).

- **Vinculación (una vez por dispositivo):** el teléfono genera un par de claves; la privada queda en su enclave seguro y en el servidor solo se guarda la **clave pública**.
- **Al fichar:** el servidor envía un desafío, el empleado lo desbloquea con su biometría, el teléfono lo firma y la Edge Function **verifica la firma** + valida la geovalla + fija la hora del servidor.
- **La biometría nunca llega al servidor** — correcto para la Ley 25.326.
- **Elimina la suplantación:** la clave privada no sale del teléfono y está protegida por la biometría de la persona.

- **Límite honesto:** el GPS es señal secundaria (falsificable en dispositivos comprometidos); el ancla fuerte es la biometría + el dispositivo vinculado.

# 8 · Listado final de tareas (por hitos)

Plan de trabajo propuesto, organizado en hitos cobrables de forma independiente. Las horas son estimadas para un desarrollador trabajando en solitario.

## Hito 0 — Auditoría y plan

**1 – 3 h**

- Finalizar y entregar el informe de auditoría.
- Confirmar el estado real de RLS con las migraciones del repositorio.
- Revisar la Edge Function: validación de rol/empresa y hora de servidor.
- Verificar que no haya una service_role key commiteada; rotarla si existe.

## Hito 1 — Backend seguro y capa de datos

**20 – 25 h**

- Completar RLS en todas las tablas con políticas por empresa_id y por rol.
- Escribir pruebas automáticas de aislamiento multiempresa.
- Mover las lecturas sensibles (empleados, sueldos, perfiles_admin, empresas) a endpoints de servidor.
- Blindar perfiles_admin y la gestión de roles del lado del servidor (no editable por el cliente).
- Fijar la hora de fichaje en el servidor y validar integridad (geovalla, duplicados, turnos abiertos).
- Generar tipos TypeScript desde el esquema de Supabase.
- Rotar y mover secretos; configurar variables de entorno.

## Hito 2 — Rebuild del frontend (Next.js)

**40 – 45 h**

- Configurar Next.js + TypeScript + Tailwind + shadcn/ui + TanStack Query.
- Definir estructura de proyecto y sistema de diseño (reutilizando el look actual).
- Autenticación con Supabase Auth + manejo de sesión y roles.
- Pantallas de administración: Dashboard, Empleados (ABM), Registros (grilla semanal, carga manual, historial de eliminados, filtros), Reportes (con exportación a Excel y PDF), Configuración (Empresa, Tarifas, Geovalla, Acceso QR, Suscripción) y Alertas.
- PWA pública de fichaje (identidad del empleado, estados, ayuda).
- Aplicar cabeceras de seguridad (CSP, HSTS, etc.).
- Diseño responsive y accesibilidad básica.

## Hito 3 — Módulo de fichaje biométrico

**15 – 20 h**

- Tabla de credenciales WebAuthn y flujo de enrolamiento por dispositivo.
- Flujo de fichaje con WebAuthn (desafío + verificación de firma del lado del servidor).
- Fallback a código/PIN del dispositivo.
- Integración con la geovalla GPS.
- Gestión desde administración: vincular, revocar y re-vincular dispositivos.
- Pruebas en dispositivos reales iOS y Android.

### Hito 4 — Performance, QA y entrega — 20 – 25 h

- Índices y optimización de consultas.
- Paginación y carga eficiente de listados.
- Optimización del bundle y de la carga inicial.
- QA funcional y de seguridad (re-test de los hallazgos).
- Dejar armados y configurados dos entornos separados: staging (para probar) y producción.
- Deploy final a producción.
- Documentación técnica y de uso.

**Qué implica dejar armados los dos entornos (staging y producción).** La idea es separar un entorno de pruebas de uno real, para poder probar cambios sin riesgo antes de que los vean los empleados. En concreto:

- Dos instancias separadas de Supabase (o ramas), cada una con su propia base, sus claves y sus datos. Los datos de prueba nunca se mezclan con los reales.
- URLs separadas para cada entorno (una de staging para probar y la definitiva de producción).
- Variables de entorno y credenciales distintas en cada uno.
- Un circuito de despliegue donde todo se prueba primero en staging y recién después pasa a producción.
- Migraciones de la base controladas: se aplican y se revisan en staging antes de tocar producción.

| TIEMPO ESTIMADO DE TRABAJO TOTAL | ≈ 118 h · aprox. 1 mes |
|---|---|

**Plazo estimado:** a razón de aproximadamente **4 h diarias de trabajo (media jornada / part-time)**, el proyecto se completaría en **aproximadamente 1 mes**.

Nota: una parte del endurecimiento de seguridad (Hito 1) se realiza durante el rebuild de la capa de datos (Hito 2), por lo que ambos esfuerzos se solapan parcialmente y no deben sumarse como costos totalmente independientes.

## 9 · Presupuesto

Presupuesto estimado para el proyecto completo (hitos 0 a 4). Para trabajos de gran volumen se aplica una tarifa preferencial por hora respecto de la tarifa estándar.

| Concepto | Detalle |
|---|---|
| Tiempo estimado de trabajo | ≈ **118 h (aprox. 1 mes, ~4 h/día)** |
| Tarifa estándar | **USD 18 / hora** |
| Tarifa aplicada al proyecto (por volumen) | **USD 12 / hora** |
| **TOTAL DEL PROYECTO** | **USD 1.416** |

Equivale a un ahorro de USD 708 respecto de la tarifa estándar (USD 6 / hora de descuento por volumen).

## Modalidad de pago

- **Recomendada:** 50% al iniciar el proyecto y 50% a la entrega final (USD 708 + USD 708).
- **Alternativa:** hasta 3 pagos distribuidos dentro del plazo de trabajo (USD 472 cada uno).
- Condiciones **negociables** según la preferencia del cliente.
- Opcional: el Hito 0 (auditoría), ya prácticamente completado, puede facturarse como primer pago.

Presupuesto estimado en dólares estadounidenses (USD). El alcance y el total definitivos se confirman tras la revisión del código fuente.